# Tabletop Traffic Junction

Observe, decide, actuate—without conflicting greens

*The signal heads reveal the state machine; the display reveals its time.*

| | | | |
|---|---|---|---|
| **Lesson** | 012 — integration project | **Time** | 180–240 minutes |
| **Prerequisites** | Lessons 001–011 | **Board** | Arduino Mega 2560 |
| **Result** | two signal heads, pedestrian pair, countdown, test points | **Status** | host verified; physical acceptance open |
| **Safety** | E1 tabletop model; USB-powered current-limited LEDs only | | |

**Safety checkpoint.** This model is not a road controller and must never direct real traffic. Disconnect USB before wiring. Every LED and display segment needs its own resistor. Use only the Mega's 5 V rail. Stop for heat, odor, resets, unexpected simultaneous greens, or disagreement between the schematic and breadboard. Remove USB power to stop the experiment.

## 1 Mission and evidence chain

Build two vehicle signal heads, a pedestrian request and signal pair, and a one-digit countdown. The project composes:

button → `TrafficInput` → `TrafficJunction` → `TrafficSignals` → current-limited LEDs

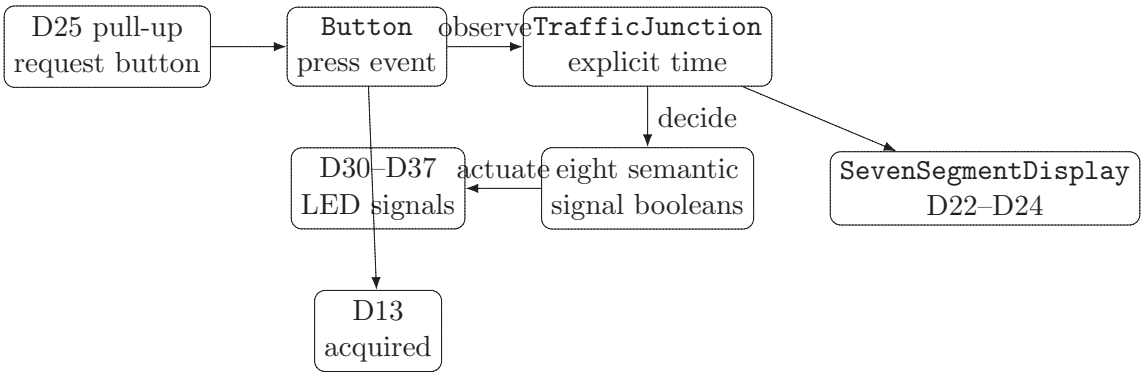
`nextDeadline` → `SevenSegmentDisplay` → `ShiftRegisterOutput` → visible digit.

The D13 board LED pulses for 250 ms only after every first-class object initializes. Initialization is transactional: a failed step shuts down every earlier object in reverse dependency order. The heads are the primary diagnostic channel. All-red plus pedestrian stop is the requested running safe state. The countdown is independent supporting evidence: it shows the remaining whole seconds until the next transition. Serial is not required. Named electrical test points are D30 (main red), D32 (main green), D33 (side red), and D35 (side green), each measured relative to GND.

By the end, you can:

- explain why a hardware-neutral state engine receives explicit time;
- trace a latched pedestrian request until it is served;
- prove from observations that both greens are never active together;
- explain why actuation turns permissive signals off before changing red;
- relate a semantic glyph to an atomic 74HC595 latch update;
- distinguish successful acquisition, a commanded all-red state, and a measured electrical all-red state.

## 2 System composition




The canonical sketch follows the same narrative. `loop()` observes one button event, asks the engine to decide, then actuates the returned snapshot. It never invents signal combinations in Arduino-facing code.

### 2.1 The state promise

| Phase | Vehicle indication | Pedestrian indication |
|---|---|---|
| `AllRed` | both red | stop |
| `MainGreen/MainYellow` | named main aspect; side red | stop |
| `SideGreen/SideYellow` | named side aspect; main red | stop |
| `PedestrianWalk` | both red | walk |
| `PedestrianClearance` | both red | stop |
| `Fault` | both red | stop |

A request is an event, not a held electrical level. The engine latches an accepted request and serves it after the side phase. Each observation also checks that every owned endpoint remains initialized. That check detects lost software ownership; it is not electrical feedback from an LED. A reported circuit-health failure latches `Fault`. The sketch then requests all-red best-effort and performs canonical shutdown. The model runs for two minutes, then releases every owned pin; reset starts a fresh run. No call blocks while a traffic phase elapses.

## 3 Wiring drawing

## 4 Parts, limits, and literal wiring

| Quantity | Part | Requirement |
|---|---|---|
| 1 | Mega 2560 | USB powered |
| 1 | 74HC595 | identified pinout; 0.1 $\mu$F local decoupling |
| 1 | one-digit display | common cathode, identified segment pinout |
| 8 | signal LEDs | two red/yellow/green heads plus red/green pedestrian pair |
| 16 | resistors | eight 330 $\Omega$ signal; eight 1 k$\Omega$ display |
| 1 | momentary button | normally open |
| 1 | multimeter | unpowered continuity and powered DC voltage |

**Mega and signal connections.**

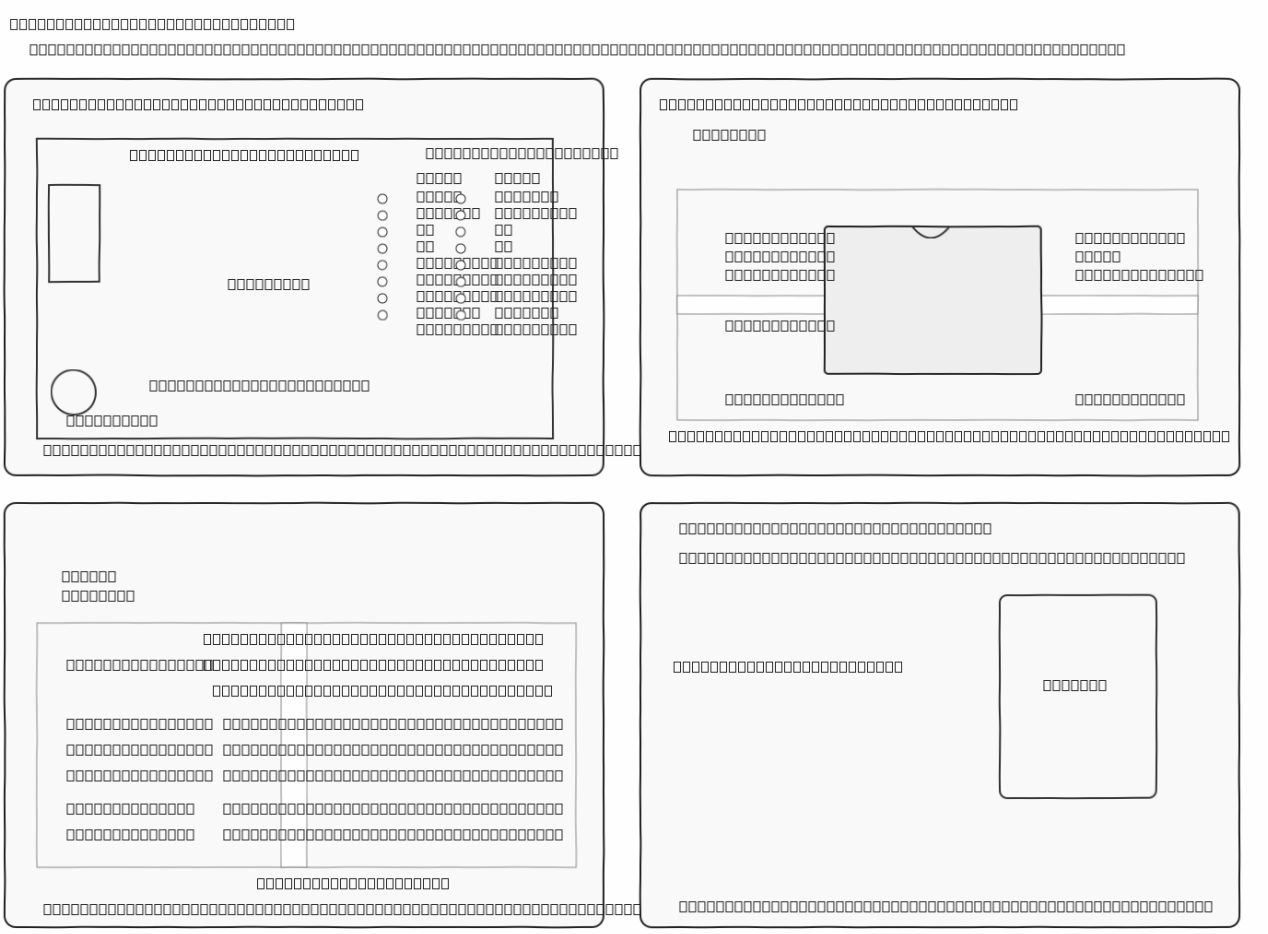


Figure 1: Four complementary pencil plates: physical Mega header locator, 74HC595 placement guide, physical signal breadboard coordinates, and specimen-gated display mapping. Match the selected display's identified package pins to the named a–g/DP nets before inserting it.

| Function | Mega pin | Literal path |
|---|---|---|
| request | D25 | D25 to button to GND; internal pull-up |
| acquisition | D13 | built-in LED; 250 ms pulse after every object initializes |
| main head | D30/D31/D32 | pin to 330 Ω to red/yellow/green anode; every cathode to GND |
| side head | D33/D34/D35 | same order and resistor rule |
| pedestrian | D36/D37 | walk green/stop red, each through 330 Ω |

**74HC595 and display connections.**

| Function | Connection | Required interpretation |
|---|---|---|
| serial data/clock/latch | D22 to DS, D23 to SHCP, D24 to STCP | `ShiftRegisterPins{22,23,24}` |
| control | MR to 5 V; OE to GND | never leave either floating; see startup limit below |
| power | VCC to 5 V; GND to GND; 0.1 $\mu$F across them | place capacitor at the IC |
| segments | Q0–Q6 to a–g, Q7 to decimal point | each path has its own 1 kΩ resistor |
| display return | both common cathodes to GND | verify the exact package pinout |

The supplied one-digit display family has multiple package mappings. Read its marking and datasheet, verify

that it is common-cathode, and write its actual pin numbers beside the named a–g/DP nets in the drawing before insertion. If those facts are unknown, stop before powering the display; the vehicle and pedestrian wins remain usable without it. With a conservative 3 mA per lit display segment and 10 mA per signal LED, the worst expected visible state is under 55 mA; the 330 Ω and 1 kΩ paths keep this USB-powered model comfortably bounded.

> **Safety checkpoint.** With `OE` tied low, the 74HC595 storage register can present an unspecified pattern before the first software latch. The three-wire lesson circuit therefore cannot promise a blank display at power-up or reset. Its atomic-update claim begins only after initialization. A later hardened adapter must own `OE` and hold it disabled with a pull-up until a known blank value has been latched.

## 5 Three quick wins

**Win 1 — make the junction breathe.** Wire the two vehicle heads and run the sketch. Both reds begin together, then permission moves through green, yellow, and all-red without ever showing two greens. If one color or direction is wrong, fix that literal D30–D35 path before adding anything else. *Expected sight:* main green with side red, then main yellow, both red, side green with main red, side yellow, and both red again.

**Win 2 — let a person ask to cross.** Add D36/D37 and the D25 button. Press and release once during main green. The later walk light is the payoff: it appears only if the button event, state engine, and both pedestrian outputs are working together. *Expected sight:* STOP stays on through the vehicle sequence; after both reds return, STOP yields to WALK, then returns.

**Win 3 — reveal the clock.** Identify the exact common-cathode display, then add the 74HC595 and one 1 kΩ resistor per segment. The digit now counts down to the same transitions already visible in the heads. A wrong segment is local to its named Q output, resistor, and display pin. *Expected sight:* one stable digit at a time, decreasing toward each lamp change without a mixed intermediate glyph.

After two minutes every lamp and the display go dark as the sketch releases its pins. Press reset and look for the short D13 acquisition pulse to play again. These wins are the learner path; deterministic traces and failure injection remain automated repository checks.

### 5.1 The narrative run loop

Listing 1: Canonical observe–decide–actuate flow.

```
    adk::SevenSegmentGlyph countdownGlyph (uint8_t seconds);
    bool                   circuitHealthy   ();
    bool                   requestAllRed    ();
    void                   shutdownJunction ();
    void                   haltJunction     ();
} // namespace

void setup ()
{
    halted = !acquireJunction ();

    if (!halted)
    {
        runStartedAt = adk::TimePoint (millis ());
    }
}

void loop ()
```

```
{
    if (halted)
    {
        return;
    }
```

The engine's `TrafficSignals` is the sole actuator model. The `showSignals()` adapter first removes green, walk, and yellow permission; then it establishes all-red; only then does it present the requested state. This ordering reduces transition hazards when writes succeed. It cannot guarantee an electrical state after a driver, wire, LED, or supply failure. The fault path attempts every all-red write even if an earlier write fails, blanks the display, and releases every endpoint to high impedance.

## 6 Watch the story unfold

| Predict | Observe | Interpret |
|---|---|---|
| Acquisition completes | D13 pulses for 250 ms | every project object initialized; this does not prove the signal pin levels |
| Startup is all-red | both red heads and pedestrian stop illuminate before any green | the first semantic state was actuated; measure test points before calling it electrical proof |
| Only one vehicle direction proceeds | watch an entire main/side cycle | green moves from one direction to the other only through yellow and all-red |
| One request is remembered | press and release D25 once during main green; wait without holding it | a later pedestrian walk proves event capture and latching |
| The display changes atomically | watch the countdown at a phase boundary | mixed or torn segments indicate wiring, resistor, or latch trouble |
| The show has a finale | let the two-minute run finish | every owned output is released and every light goes dark; reset starts another show |

The lamps explain the code while it runs: the D13 pulse introduces the show, the two heads reveal legal traffic motion, the pedestrian pair reveals a remembered request, and the digit reveals when the next change will happen.

## 7 Try these variations

Once all three wins work, predict what the model will do before each variation, then watch the lamps and digit:

1. Press once during main green and guess when walk will appear.
2. Press several times during one phase; the crossing remains one orderly request rather than skipping ahead.
3. Reset during green. The next visible traffic state is all-red, preceded by the short D13 pulse.
4. Let the two-minute show finish. Everything goes dark; reset is the only way to begin another run.

## 8 Diagnosis

| Observation | Likely layer | Next bounded check |
|---|---|---|
| No LEDs and blank display | power or acquisition | remove USB; inspect rails, polarity, pin map, and resource conflicts |
| Heads correct, digit wrong | display encoding/wiring | run lesson 010; compare Q0–Q7 with a–g/decimal point |
| Digit tears during change | latch wiring | inspect STCP D24 and shared ground; do not add delays to hide it |
| Request never served | button/event path | observe raw D25 and repeat lesson 003 debounce evidence |
| Both greens visible | actuator wiring or severe defect | remove USB immediately; unpowered continuity from D32/D35 to the intended LEDs |
| All-red forever | startup/fault/configuration | reproduce the same timestamps in host tests and inspect `status` |
| Unexpected resets | current or wiring | remove USB; measure resistors and inspect for rail shorts before reapplying power |

## 9 Make it yours

1. Make cardboard signal-head masks so each LED shines through its own round opening.
2. Change only the configured phase durations and invent a fast “tiny town” cycle or a leisurely “parade day” cycle.
3. Show a dash during all-red and digits during moving phases.
4. Add a second cardboard pedestrian button cap labelled *WAIT*; keep both caps on the same safe D25 switch rather than adding wiring.

## 10 What to carry forward

The reusable idea is not “a traffic light.” It is the boundary between a deterministic decision and a transactional physical presentation. The engine speaks in domain signals; endpoint-owning adapters perform safe ordering; the circuit supplies evidence alongside its useful behavior. The same separation supports later environmental, access, rover, greenhouse, telemetry, and inert cue projects.

**Primary source paths:** Traffic junction API, seven-segment API, shift-register API, and canonical example.